Equipo 5, S.A. de C.V.

ADRIÁN TAME JACOBO
ALEJANDRO MUÑOZ GUTIÉRREZ
MIGUEL CALVO VALENTE
NELSON GIL VARGAS
RODRIGO JUÁREZ JARAMILLO

Problema

1. SOMOS UNA EMPRESA QUE OFRECE SERVICIO DE CONSULTORÍA A LA INDUSTRIA DEL CINE Y LA TELEVISIÓN. EL CLIENTE ES UN IMPORTANTE PRODUCTOR QUE QUIERE FILMAR UNA PELÍCULA DE LA CUAL OBTENGA GANANCIAS Y QUE GUSTE AL PÚBLICO.

2. UN SITIO WEB BUSCA DESARROLLAR UN ALGORITMO QUE RECOMIENDE PELÍCULAS A NUEVOS USUARIOS.

Qué pide el cliente:

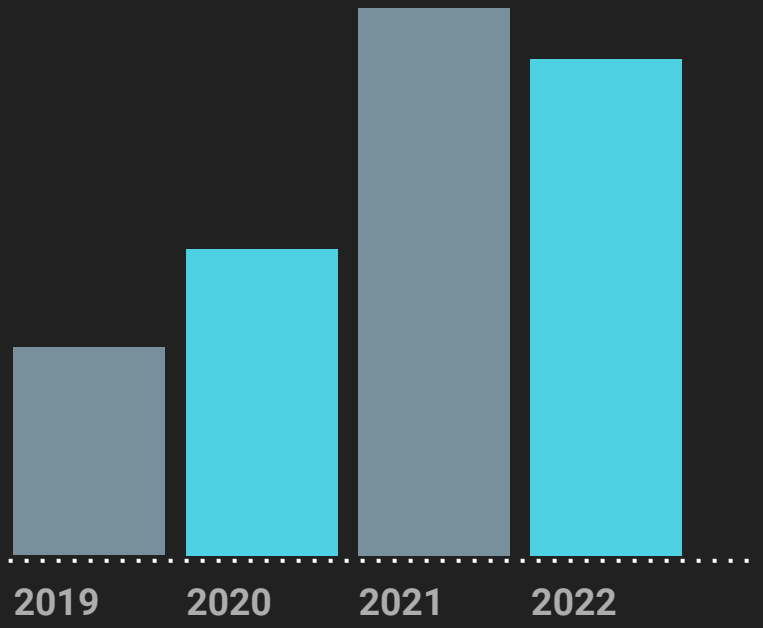
El cliente requiere un modelo que le ayude a recomendar películas que podrían gustar a nuevos usuarios.

Una manera de estimar ganancias al producir una película con ciertas características.

Qué enfrentaremos:

En caso de existir un cliente nuevo, no tendremos forma de comparar sus gustos y preferencias, por lo cual, no podríamos hacer una recomendación con base en sus gustos, historial de búsqueda y contenido de la película.

Dadas ciertas variables, determinar las ganancias de una película.



OBJETIVOS

1. Para resolver el primer problema, buscamos saber cuáles son las películas más exitosas y hacer un análisis de las cualidades que tiene una buena película. Queremos hacer análisis con base en los directores, presupuesto, género, duración y lenguaje, como algunos ejemplos de variables de interés, para predecir ganancias finales.
2. Idealmente, queremos caracterizar o agrupar los usuarios, para que cuando llegue un nuevo usuario, lo podamos asignar a uno de estos grupos, y recomendarle películas. Para esto, seguramente empezaremos con clustering, pero tal vez acabemos con otro modelo. Además, nos gustaría entender las tendencias de ciertos grupos que ven películas.

Organización de los datos

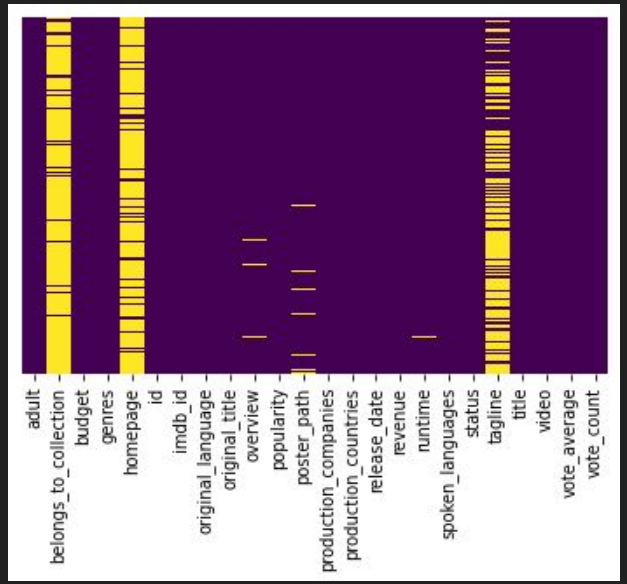


Recomendación películas

Para el Recomendador: The Movies Dataset

#	Column	Non-Null Count	Dtype
0	adult	45463 non-null	object
1	belongs_to_collection	4491 non-null	object
2	budget	45463 non-null	object
3	genres	45463 non-null	object
4	homepage	7779 non-null	object
5	id	45463 non-null	int64
6	imdb_id	45446 non-null	object
7	original_language	45452 non-null	object
8	original_title	45463 non-null	object
9	overview	44509 non-null	object
10	popularity	45460 non-null	float64
11	poster_path	45077 non-null	object
12	production_companies	45460 non-null	object
13	production_countries	45460 non-null	object
14	release_date	45376 non-null	object
15	revenue	45460 non-null	float64
16	runtime	45203 non-null	float64
17	spoken_languages	45460 non-null	object
18	status	45379 non-null	object
19	tagline	20412 non-null	object
20	title	45460 non-null	object
21	video	45460 non-null	object
22	vote_average	45460 non-null	float64
23	vote_count	45460 non-null	float64

Tenemos 45,463 películas, de los años 1874 a 2020.



Variables de interés: The Movies Dataset / EDA recap

Numéricas

	budget	popularity	revenue	runtime
0	30000000	21.946943	373554033.0	81.0
1	65000000	17.015539	262797249.0	104.0
2	0	11.712900	0.0	101.0
3	16000000	3.859495	81452156.0	127.0
4	0	8.387519	76578911.0	106.0

Categóricas

Género

	id	name	parent_id
0	16.0	Animation	0
1	35.0	Comedy	0
2	10751.0	Family	0
3	12.0	Adventure	1
4	14.0	Fantasy	1

Lenguajes

	iso_639_1	name	parent_id
0	en	English	0
1	en	English	1
2	fr	Français	1
3	en	English	2
4	en	English	3

Para modelo de predicción con ratings: small ratings

Para utilizar el modelo del paquete Surprise de Python, utilizamos la base de datos de ratings, la cual contiene el userID, movieID, ratings y timestamp, la cual, contiene 671 usuarios únicos, con 9066 diferentes películas. Los ratings van de .5 a 5 estrellas.

```
reader = Reader()
ratings.head()

ratings['Time'] = ratings['timestamp'].apply(datetime.fromtimestamp)
print(ratings)
```

```
        userId  movieId  rating   timestamp                Time
0            1       31     2.5  1260759144 2009-12-14 02:52:24
1            1     1029     3.0  1260759179 2009-12-14 02:52:59
2            1     1061     3.0  1260759182 2009-12-14 02:53:02
3            1     1129     2.0  1260759185 2009-12-14 02:53:05
4            1     1172     4.0  1260759205 2009-12-14 02:53:25
...        ...      ...     ...         ...                 ...
99999      671     6268     2.5  1065579370 2003-10-08 02:16:10
100000     671     6269     4.0  1065149201 2003-10-03 02:46:41
100001     671     6365     4.0  1070940363 2003-12-09 03:26:03
100002     671     6385     2.5  1070979663 2003-12-09 14:21:03
100003     671     6565     3.5  1074784724 2004-01-22 15:18:44

[100004 rows x 5 columns]
```

```
[17] len(pd.unique(ratings['userId']))

671

[18] len(pd.unique(ratings['movieId']))

9066
```

```
ratings.describe()
```

	userId	movieId	rating	timestamp
count	100004.000000	100004.000000	100004.000000	1.000040e+05
mean	347.011310	12548.664363	3.543608	1.129639e+09
std	195.163838	26369.198969	1.058064	1.916858e+08
min	1.000000	1.000000	0.500000	7.896520e+08
25%	182.000000	1028.000000	3.000000	9.658478e+08
50%	367.000000	2406.500000	4.000000	1.110422e+09
75%	520.000000	5418.000000	4.000000	1.296192e+09
max	671.000000	163949.000000	5.000000	1.476641e+09

Para el modelo de predicción con base en el contenido de la película

Para utilizar el modelo de contenido, utilizamos la base de datos de credits y movies.
Las variables importantes son movieID, title y overview.
La variable overview tiene el contenido general de cada película.

```
df2['overview']
```

```
0       In the 22nd century, a paraplegic Marine is di...
1       Captain Barbossa, long believed to be dead, ha...
2       A cryptic message from Bond’s past sends him o...
3       Following the death of District Attorney Harve...
4       John Carter is a war-weary, former military ca...
                              ...
4798    El Mariachi just wants to play his guitar and ...
4799    A newlywed couple's honeymoon is upended by th...
4800    "Signed, Sealed, Delivered" introduces a dedic...
4801    When ambitious New York attorney Sam is sent t...
4802    Ever since the second grade when he first saw ...
Name: overview, Length: 4803, dtype: object
```

Train y Test sets

Para utilizar el modelo de recomendación del paquete surprise, SVD y KNN, utilizamos validación cruzada 5.

```
#Leemos la base de datos para poder ser utilizada por la librería
data = Dataset.load_from_df(ratings[['userId', 'movieId', 'rating']], reader)
# Utilizamos el algoritmo de SVD: Singular Value Decomposition, que sirve para hacer modelos predictivos.
algorithm_1 = SVD()
trainset, testset = train_test_split(data, test_size=.30)
# Hacemos validación cruzada 5 para medir el error de testing
cross_validate(algorithm_1, data, measures=['RMSE', 'MAE'], cv=5, verbose=True)
```

```
Evaluating RMSE, MAE of algorithm SVD on 5 split(s).

                  Fold 1  Fold 2  Fold 3  Fold 4  Fold 5  Mean    Std
RMSE (testset)    0.8965  0.9045  0.8928  0.8939  0.8931  0.8962  0.0044
MAE (testset)     0.6907  0.6943  0.6892  0.6879  0.6861  0.6896  0.0028
Fit time          5.27    5.23    5.31    5.37    5.15    5.26    0.07
Test time         0.18    0.43    0.18    0.17    0.19    0.23    0.10
```

```
Evaluating RMSE, MAE of algorithm SVD on 5 split(s).

                  Fold 1  Fold 2  Fold 3  Fold 4  Fold 5  Mean    Std
RMSE (testset)    0.8920  0.8910  0.9025  0.8930  0.9007  0.8958  0.0048
MAE (testset)     0.6883  0.6877  0.6944  0.6887  0.6908  0.6900  0.0024
Fit time          5.45    5.50    5.19    5.32    5.20    5.33    0.13
Test time         0.20    0.30    0.22    0.28    0.17    0.24    0.05
```

Para el Recomendador

K-Means

Se ocupó el 100% del conjunto de películas como conjunto de entrenamiento.

Overview

Se ocupó el 100% del conjunto de películas como conjunto de entrenamiento.



Propuesta de métodos

¿Qué métodos se proponen para resolver el problema?

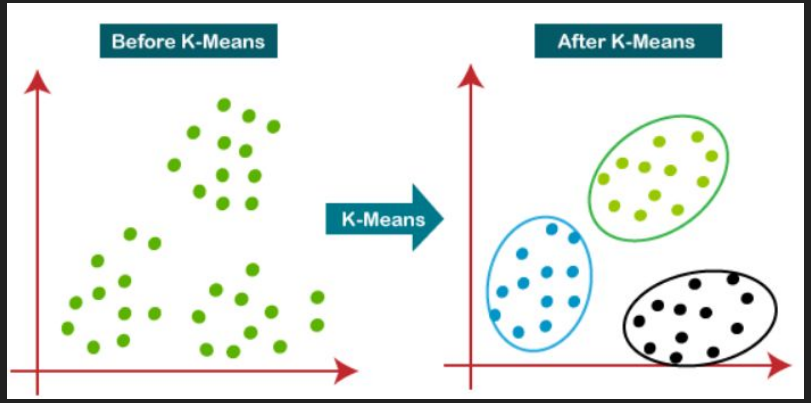
Para el Recomendador

Ocupamos un modelo de clustering con K-Means con base en géneros, lenguajes y algunas variables numéricas.

Para conocer la similitud del contenido de las películas, utilizamos análisis de texto y medimos la distancia que había entre el contenido de una película y otra.

Para recomendar películas con base en el rating otorgado de los usuarios, utilizamos Single Value Decomposition.

K-Means



SVD

	Inside Out	Good Will Hunting	Mean Girls	Terminator	Titanic	Warrior
Tina Fey	3	1	5	1	?	1
Helen Mirren	2	?	?	2	5	1
Sylvester Stallone	1	3	1	4	2	5
Tom Hanks	?	3	1	?	4	3
George Clooney	2	2	1	3	1	4



Resultados preliminares

Resultados Preliminares modelo SVD vs KNN:

Hicimos validación cruzada para evaluar el Error Absoluto Medio y el Error Cuadrático Medio.

Evaluating RMSE, MAE of algorithm SVD on 5 split(s).

	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Mean	Std
RMSE (testset)	0.8920	0.8910	0.9025	0.8930	0.9007	0.8958	0.0048
MAE (testset)	0.6883	0.6877	0.6944	0.6887	0.6908	0.6900	0.0024
Fit time	5.45	5.50	5.19	5.32	5.20	5.33	0.13
Test time	0.20	0.30	0.22	0.28	0.17	0.24	0.05

Evaluating RMSE, MAE of algorithm KNNBasic on 5 split(s).

	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Mean	Std
RMSE (testset)	0.9666	0.9721	0.9723	0.9770	0.9539	0.9684	0.0080
MAE (testset)	0.7427	0.7487	0.7467	0.7521	0.7321	0.7445	0.0069
Fit time	0.19	0.23	0.20	0.20	0.21	0.21	0.02
Test time	2.13	1.97	1.94	1.93	1.87	1.97	0.09

Resultados Preliminares modelo SVD vs KNN:

```
[24] # Obtenemos una predicción de rating dependiendo de una película
     #En este caso vamos a predecir del UserID 40, qué rating le daría a la película 302.
     pred_1 = algorithm_1.predict(uid=40, iid=302, r_ui=3)
     print(pred_1)
```

```
user: 40         item: 302        r_ui = 3.00   est = 4.19   {'was_impossible': False}
```

Para este caso, el modelo de SVD() predice que el cliente 40 o usuario 40 le otorgará una calificación de 3.84 a la película *302* con un error promedio de .69

```
[26] #En este caso vamos a predecir del UserID 40, qué rating le daría a la película 302.
     pred_2 = algorithm_2.predict(uid=40, iid=302, r_ui=3)
     print(pred_2)
```

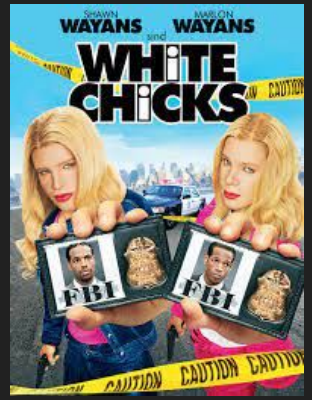
```
user: 40         item: 302        r_ui = 3.00   est = 3.49   {'actual_k': 4, 'was_impossible': False}
```

Para este caso, el modelo de KNN() predice que el cliente 40 o usuario 40 le otorgará una calificación de 3.49 a la película *302* con un error promedio de .744

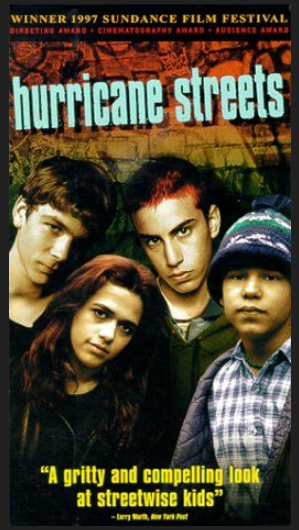
Resultados preliminares

```
get_recommendations('White Chicks')
```

```
2511                                  Home Alone
186                                   Bad Boys II
4529                          Hurricane Streets
753                                   The Sentinel
2269                        Butterfly on a Wheel
2086    Home Alone 2: Lost in New York
4726                                  The Mighty
1422    The X Files: I Want to Believe
731                                   The Jackal
2813                         Think Like a Man
Name: title, dtype: object
```



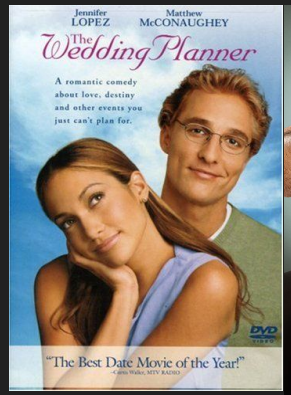




Resultados Preliminares:

K-Means: Similares a White Chicks

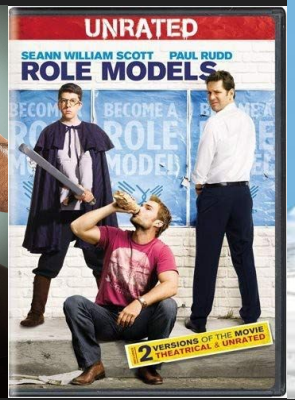




kraft macaroni and cheese; therapist; love triangle



london england; misunderstanding; vomiting



treference to dungeons and dragons; sex; stripping



cogresswoman; the white house; animal rights



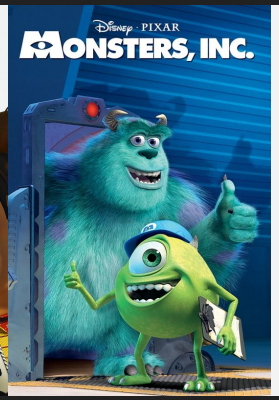
puppet; miss piggy; satirical

K-Means: Toy Story





toy; toy comes to live; animation



monster world; human nonhuman relationship



the simpsons; pig; lake; dome



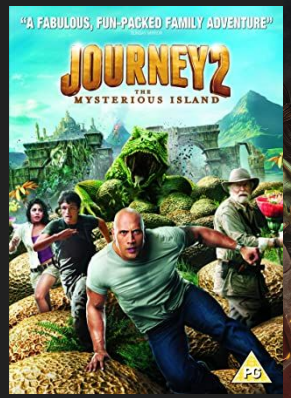
chicken; alien; panic



chicken; farm; escape

K-Means: Similares a Transformers: Dark of the Moon





lost city; atlantis; lost world



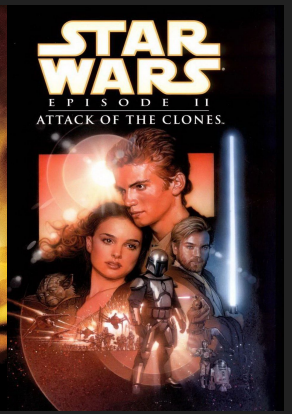
based on videogame; orc; fictional language



personality lone whange; elongated cry of no; tragic villain



time travel; post apocalypse; abduction

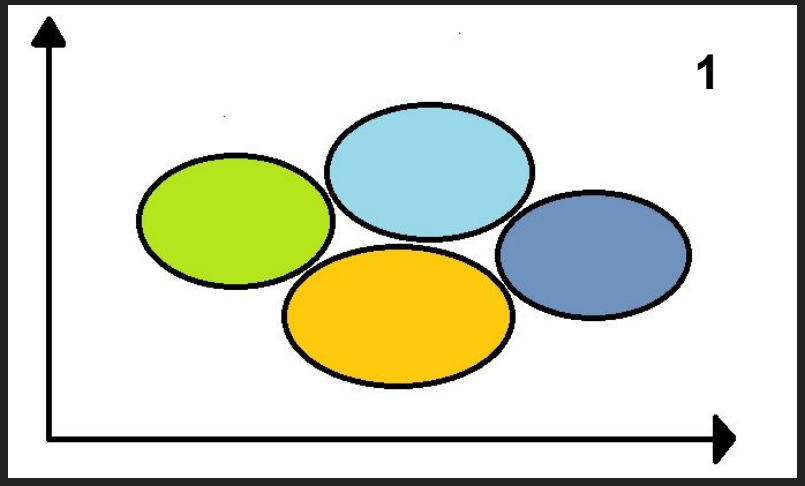


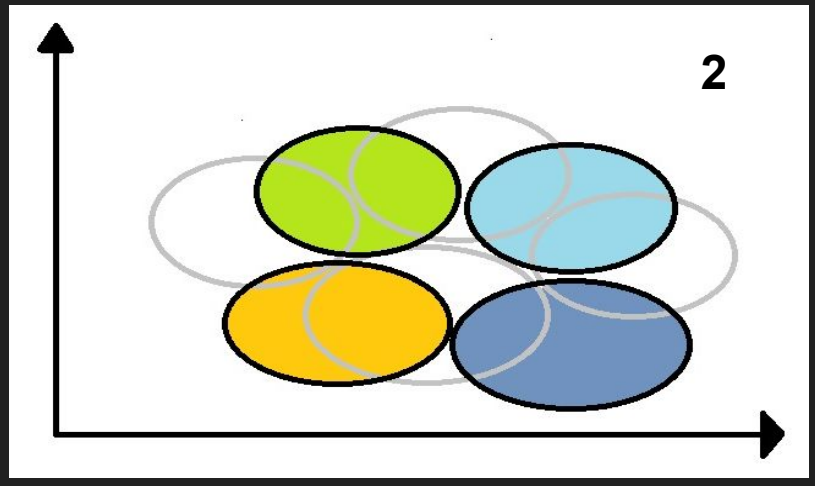
clone wars; the force; part of trilogy

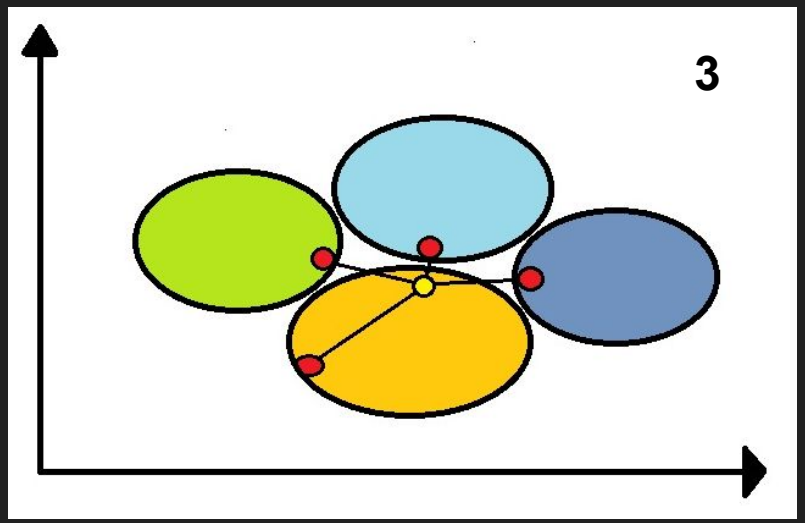
Siguientes pasos

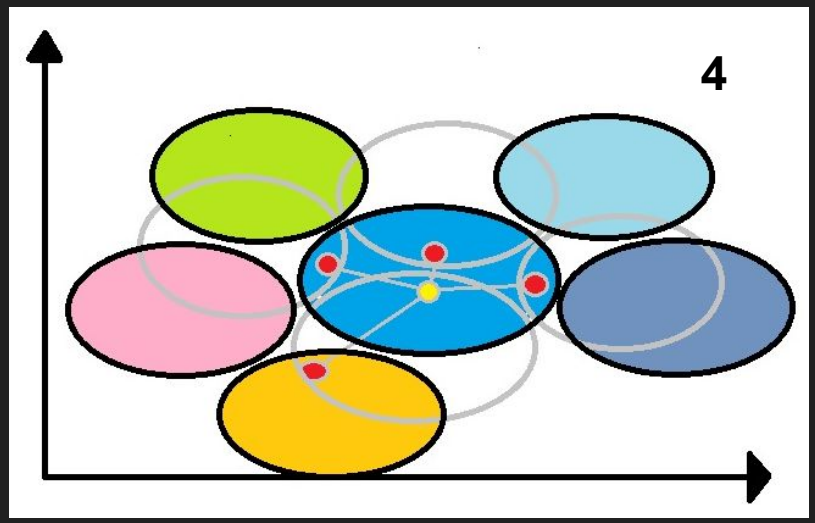
Por hacer:

- K - Means
  - Ocupar M corridas de K-Means
  - Ocupar diferentes niveles de K, e ir sugiriendo de mayor a menor

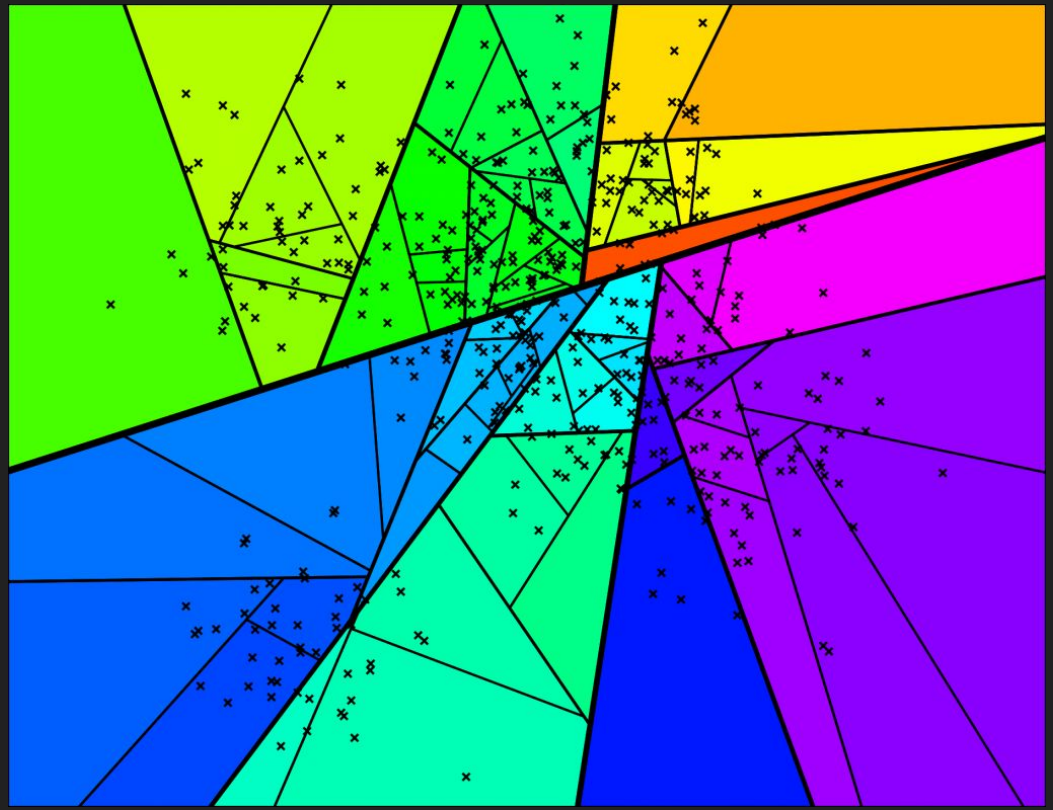

1


2


3


4

Approximate Nearest Neighbors



Siguientes pasos

Por hacer:

- Overview
  - Podríamos utilizar overview como una variable explicativa a un modelo más robusto.

- SVD
  - Incluir el título de las películas en las predicciones.

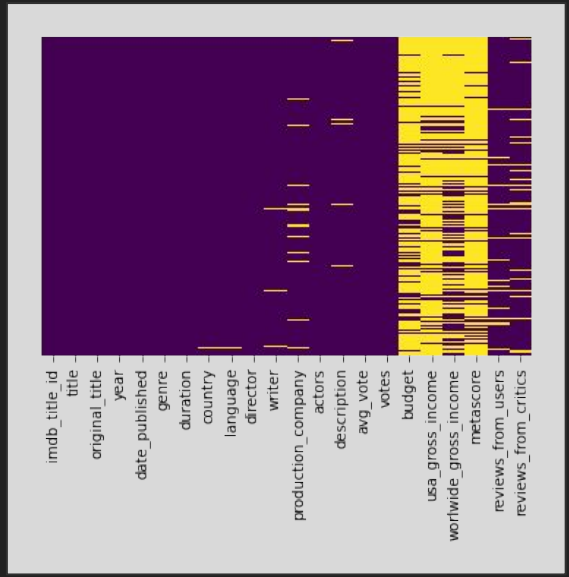
- Más ideas
  - Generar una métrica de qué tan “importante” es un actor y/o director, y sugerir películas con base en si hay actores y/o directores “importantes”
  - Con todos los modelos e ideas, incluir la probabilidad que recomiende uno o de otro.

Predicción de Ganancias

Para Regresión: Base IMDb

#	Column	Non-Null Count	Dtype
0	imdb_title_id	85855 non-null	object
1	title	85855 non-null	object
2	original_title	85855 non-null	object
3	year	85855 non-null	object
4	date_published	85855 non-null	object
5	genre	85855 non-null	object
6	duration	85855 non-null	int64
7	country	85791 non-null	object
8	language	85022 non-null	object
9	director	85768 non-null	object
10	writer	84283 non-null	object
11	production_company	81400 non-null	object
12	actors	85786 non-null	object
13	description	83740 non-null	object
14	avg_vote	85855 non-null	float64
15	votes	85855 non-null	int64
16	budget	23710 non-null	object
17	usa_gross_income	15326 non-null	object
18	worlwide_gross_income	31016 non-null	object
19	metascore	13305 non-null	float64
20	reviews_from_users	78258 non-null	float64
21	reviews_from_critics	74058 non-null	float64

Tenemos 85,855 películas, de los años 1894 a 2020.
Tomamos todos los renglones que no tienen NA (~6600).



Variables de interés: IMDB / EDA recap

Variables Explicativas

- Duración [Continua]
- Voto Promedio [Continua]
- Número de Votos [Integer]
- Presupuesto [Continua]
- Metascore (voto ponderado entre usuarios, críticos, y super-usuarios) [Continua]
- Reviews de Usuarios [Integer]
- Reviews de Críticos [Integer]
- Género de la película [Categórica]
- Lenguaje [Categórica]
- País [Categórica]
- Director [Categórica]
- Escritor [Categórica]
- Compañía de Producción [Categórica]

895 variables de entrenamiento

Variables Objetivo

- Gross Income [Continua]
  - USA Gross Income + Worldwide Gross Income [Continuas]
  - Transformada con logaritmo



Propuesta de métodos

¿Qué métodos se proponen para resolver el problema?

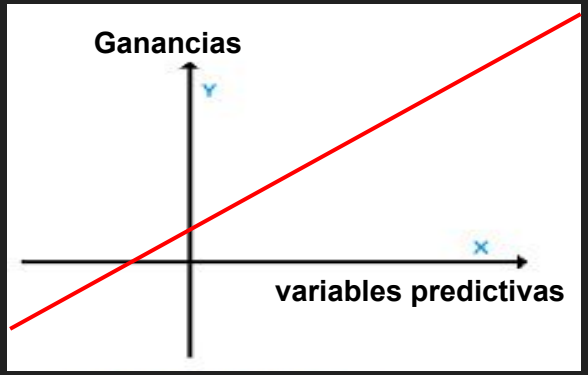
(Justificar los métodos que se van a utilizar)

Para Regresión: Base IMDb

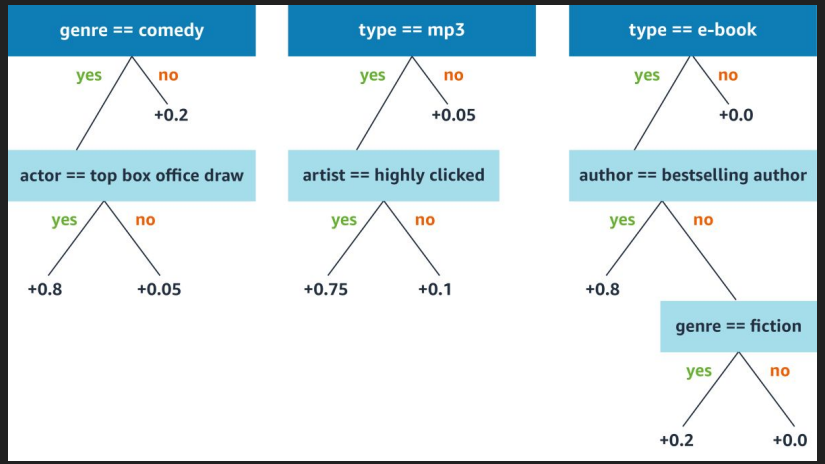
Usamos regresión, y decidimos concentrarnos en 4 distintos modelos:

- kNN
- Decision Tree
- Regresión Regularizada a través de Lasso
- SVR

Regresión Regularizada a través de Lasso



Decision Tree





Resultados preliminares

Resultados Preliminares:

¿Desempeño inicial? (Interpretación de R cuadrada)

kNN	Performance on the training set: 0.735 Performance on the validation set: 0.621
Regresión Lasso	Best score: 0.554 Best params: {'alpha': 0.0005672289891977495, 'normalize': 'True'}
Decision Tree Regression	Performance on the training set: 1.000 Performance on the validation set: 0.508
Support Vector Regression	Performance on the training set: 0.472 Performance on the validation set: 0.455

Siguientes pasos

Por hacer:

- Separar a archivos diferentes la limpieza del entrenamiento
- Tuning de modelos
- Más modelos
- Interacciones

Tal Vez:

- Variable actores
- Variable fecha a día de la semana

Referencias:

- https://www.amazon.science/blog/speeding-training-of-decision-trees
- Movie Recommendation System
- Surprise, Python
- Surprise, Python: getting started